Coding interview companion

Sreedhar Suswaram | Senior Software Engineer | American Express technical and coding round | October 23, 2026

This is the coding section of your preparation in one place. It fits **inside** the 20 hours in the main guide: hours 1, 4-6, 16-17, and 19 provide seven dedicated coding hours. Use one hour of the Java study blocks (hours 2-3) for the mixed drill below; the hour-18 mock also includes coding. Use Java unless the interviewer asks for another language. These exercises are tailored practice, not confirmed interview questions from American Express.

Your goal: solve a correct problem in a way another engineer can follow. A good interview answer includes assumptions, a working approach, a short correctness argument, complexity, and tests. You do not need to speak continuously while typing; use short updates at decision points.

1. Your eight-hour coding path

Block	Time	Work	Pass condition
Baseline	1 h	Resume introduction, Two Sum from a blank editor, one SQL query, and review	State constraints, code, trace, test, and complexity
Core patterns A	1 h	Hash maps, grouping, sets; problems 1 and 2	Pick a key and explain its equality rules
Core patterns B	1 h	Sliding window, two pointers; problems 3 and 4	State the invariant and move pointers correctly
Core patterns C	1 h	Sorting/intervals and binary search; problems 5 and 6	Handle ties, boundaries, and empty input
Mock 1	1 h	Unseen problem: longest substring with at most K distinct characters	Solve in 35 min; test and explain in 10 min
Mock 2	1 h	Payment totals by account and currency, then one SQL query	Preserve money and tenant rules; finish cleanly
Mixed drill	1 h	Use part of Java study hours 2-3 for stack, heap, BFS, and a debugging prompt	Recognize correct data structure within 2 min
Repair	1 h	Repeat weakest problem from a blank editor, then 15-min oral drill	Solve without looking; state two things learned

If you are short of time, do problems 1-6 plus the two mocks. Problems 7-12 are valuable after the essentials. Keep breaks outside these focused hours. The final hour before the interview should be gentle recall rather than a new hard problem.

2. A repeatable live-coding method

Step 1: Clarify the contract (2-3 minutes)

Ask what the input can contain and what the output must mean. Clarify null or empty input, duplicates, sortedness, negative values, integer overflow, case sensitivity, Unicode, and tie behavior **only when relevant**. If the interviewer leaves a choice open, state your assumption and proceed.

Say: "Let me confirm the output and a couple of edge cases. Then I'll walk through an example before coding."

Example for Two Sum: "Should I return indices or values? Can the same number occur twice? I assume I must use two different indices. What should I return if there is no pair?"

Step 2: Trace a small example (2 minutes)

Choose an example that reveals the rule, not just the happy path. For Two Sum, `[3, 3]` with target 6 reveals whether two identical values at distinct indices are allowed. For interval merging, `[1,10], [2,3], [10,12]` reveals nesting and touching endpoints.

Say: "On this example, the expected result is ... because ..."

Step 3: Choose a simple approach and improve it (2-4 minutes)

Name the straightforward solution first when it helps. State why a data structure improves it. Do not recite every possible algorithm.

Say: "A nested scan would take O(n squared) time. A map of values already seen lets me check each complement once, so I expect O(n) time and O(n) extra space."

Step 4: State the invariant (1 minute)

An invariant is a rule that stays true after every loop iteration. It is the core of a correctness explanation. Say it before a complicated loop.

- Hash map: "The map contains only elements before the current index."
- Sliding window: "The current window contains at most K distinct characters."
- Binary search: "The answer, if it exists, remains in the search range, or I have already saved a candidate."
- Merge intervals: "The result contains merged intervals for everything processed so far."

Step 5: Code in small, complete pieces (15-25 minutes)

Write a method signature, initialize the structures, write the main loop, return the result. Use clear names such as `left`, `right`, `counts`, `lastSeen`, and `merged`. Do not add a framework, helper classes, or streams when a small loop is clearer. In Java, choose `ArrayDeque` for stack/queue practice and a suitable `Map` or `Set` for lookup.

Say briefly while coding: "I am checking the complement before storing the current item so I cannot use the same index twice."

Step 6: Trace and test (5-8 minutes)

Run the small example through the code. Then use at least one boundary case and one adversarial case. Check whether the input is mutated. If you find a bug, correct it openly and test again.

Say: "I'll test empty input, duplicate values, and the boundary where the answer is at the end. I found a boundary issue; I'll fix it and retrace."

Step 7: Close the answer (1-2 minutes)

Give time and extra-space complexity, mention the dominant operation, and state one important limitation. If you used sorting, include sorting cost. If you created an output array, say whether your space figure includes output.

Say: "The scan is O(n) expected time because each hash lookup is expected constant time. The map uses O(n) additional space. A hash collision does not change correctness, although worst-case behavior can differ."

3. Recognize the pattern before you code

Clue in prompt	Pattern	Main idea	Common mistake
Find pair or repeated key	Hash map/set	Record what was seen	Use same index twice; forget key equality
Longest/shortest contiguous range	Sliding window	Maintain a valid window	Move left backward; fail to shrink fully
Sorted array and first/last match	Binary search	Cut search space in half	Overflow in midpoint; wrong loop bounds
Overlapping time or number ranges	Sort + sweep	Merge with last interval	Replace end instead of taking max
Matching brackets or nested syntax	Stack	Last opened is first closed	Ignore extra closing bracket
Top K or next minimum/maximum	Heap	Keep only useful candidates	Wrong heap ordering or tie behavior
Levels or shortest unweighted path	BFS queue	Visit distance layer by layer	Mark visited too late; revisit nodes
All paths or recursive structure	DFS	Explore one branch at a time	Missing base case; shared mutable state
Number of subarrays with target sum	Prefix sum + map	Count prior required prefix values	Initialize count of zero prefix incorrectly
Repeated subproblems with choices	Dynamic programming	Store answers to smaller states	State not defined before recurrence

An interviewer may use a payment or account story around a standard pattern. Translate the business words into the data structure problem. Also preserve business rules: group money by currency, scope accounts by tenant, and do not treat a local cache as a durable deduplication mechanism.

Complexity you should say without hesitation

Operation	Typical time	Important qualifier
Traverse n items	O(n)	Nested traversals can be O(n squared)
Sort n items	O(n log n)	Include sorting in a merge-interval solution
HashMap get/put	Expected O(1)	Hashing/equality costs and worst cases matter
TreeMap get/put	O(log n)	Maintains sorted keys
Binary search	O(log n)	Input must already be sorted appropriately
Heap offer/poll	O(log n)	Peek is O(1); build and final sorting have costs
BFS/DFS	O(V + E)	Plus visited storage for a graph
Sliding window	Usually O(n)	Each pointer moves at most n times

For a loop with one pointer moving forward and another pointer that never moves backward, do not assume O(n squared) simply because the loops are nested. Count the total pointer movements.

4. Worked problem A: Two Sum

Contract: Given `int[] values` and `int target`, return two distinct indices whose values add to target. Return an empty array if none exists. Null input is invalid. If more than one pair exists, the first found by scanning left to right is acceptable.

Walkthrough: Input `[2, 7, 11, 15]`, target 9. At 2, seek 7; it is not in the map, so store 2 -> 0. At 7, seek 2; find index 0, so return `[0,1]`.

Code:

```
static int[] twoSum(int[] values, int target) {
    Objects.requireNonNull(values);
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < values.length; i++) {
        long need = (long) target - values[i];
        if (need >= Integer.MIN_VALUE && need <= Integer.MAX_VALUE) {
            Integer previous = seen.get((int) need);
            if (previous != null) return new int[] {previous, i};
        }
        seen.put(values[i], i);
    }
    return new int[0];
}
```

Why it works: Before index `i`, the map contains only earlier indices. A found complement therefore gives a different index. If no complement is found, adding the current value makes it available to later elements. Every possible earlier partner is checked when the later partner arrives.

Tests: `[3,3]`, target 6 -> `[0,1]`; `[]` -> `[]`; `[1,2]`, target 9 -> `[]`; `[-5,2,7]`, target 2 -> `[0,2]`; use an overflow case such as two `Integer.MAX_VALUE` values with target `-2` and expect no false match.

Cost: O(n) expected time; O(n) extra space. The `long` calculation prevents integer wraparound from creating a false complement.

Professional explanation: "I store earlier values in a map from value to index. At each position I first check whether the value needed to reach the target is already present. Checking before insertion ensures I use two distinct indices. I also calculate the complement in a wider type to avoid integer overflow."

5. Worked problem B: longest substring with at most K distinct characters

Contract: Given a nonnull Java `String` and integer `k >= 0`, return the length of its longest contiguous substring containing at most K distinct UTF-16 `char` values. This is a practice problem; clarify Unicode handling in a real interview.

Walkthrough: For `"eceba"`, `k = 2`, the best window is `"ece"` with length 3. When `b` enters, three distinct letters are present. Move `left` until the window again has at most two distinct letters.

Code:

```
static int longestAtMostKDistinct(String s, int k) {
    Objects.requireNonNull(s);
    if (k < 0) throw new IllegalArgumentException("k must be nonnegative");
    Map<Character, Integer> counts = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char added = s.charAt(right);
        counts.merge(added, 1, Integer::sum);
        while (counts.size() > k) {
            char removed = s.charAt(left++);
            int remaining = counts.get(removed) - 1;
            if (remaining == 0) counts.remove(removed);
            else counts.put(removed, remaining);
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Why it works: The counts describe exactly the current window. If adding a character makes the window invalid, moving `left` removes characters until it becomes valid. At each `right`, the resulting window is the longest valid window ending there. Taking the maximum across all right endpoints finds the answer.

Tests: `"eceba", 2 -> 3`; `"aa", 1 -> 2`; `"abc", 0 -> 0`; `"", 3 -> 0`; `"abc", 5 -> 3`; negative k -> exception. For full Unicode code points, use a code-point based representation instead of `char` indexing.

Cost: O(n) time because each pointer moves forward at most n times; O(min(n, character set size)) extra space.

Professional explanation: "I maintain a window and a count for each character inside it. I expand the right side and shrink the left side only when we exceed K distinct characters. Both pointers move forward, so the total work is linear."

6. Worked problem C: merge intervals

Contract: Merge overlapping **closed** intervals `[start,end]`. Touching intervals such as `[1,2]` and `[2,3]` overlap. Reject malformed intervals. Return new arrays rather than modifying the input.

Walkthrough: Sort `[8,10], [1,3], [2,6], [15,18]` to `[1,3], [2,6], [8,10], [15,18]`. Merge the first two into `[1,6]`, then start new intervals for the rest.

Core loop after validating and sorting:

```
List<int[]> merged = new ArrayList<>();
for (int[] next : sorted) {
    if (merged.isEmpty() || next[0] > merged.get(merged.size() - 1)[1]) {
        merged.add(next.clone());
    } else {
        int[] current = merged.get(merged.size() - 1);
        current[1] = Math.max(current[1], next[1]);
    }
}
```

Why it works: Sorting means no future interval can start earlier than the current one. The last merged interval either overlaps `next` or finishes before it. Taking the maximum end handles nested intervals such as `[1,10]` followed by `[2,3]`.

Tests: Empty input; single interval; unsorted input; nesting; touching endpoints; invalid start greater than end. State whether time intervals are closed or half-open, because that changes the overlap test.

Cost: O(n log n) time for sorting; O(n) additional space when copying input and constructing output.

Professional explanation: "I sort by start time, then scan once. When the next interval overlaps the last merged interval, I extend its end to the larger endpoint. Otherwise I begin a new merged interval. The sort dominates the running time."

The full, runnable implementations for all ten core problems remain in **InterviewPractice.java**. Complete the exercise before using that file as an answer key.

7. Java coding tools to know by memory

Need	Java choice	One thing to remember
Fast lookup	`HashMap<K,V>`, `HashSet<T>`	Correct equals/hashCode for custom keys
Preserve insertion order	`LinkedHashMap<K,V>`	Insertion order is different from access order
Sorted keys	`TreeMap<K,V>`	O(log n) operations
Stack or queue	`ArrayDeque<T>`	`push/pop`, `add/remove`; no null elements
Top K / next item	`PriorityQueue<T>`	Smallest by comparator is at head
Text construction	`StringBuilder`	Build in a loop, convert once
Sorting objects	`Comparator.comparing(...)`	Use `Integer.compare`, not subtraction, for integer comparison
Exact decimal money	`BigDecimal`	Construct from a decimal string and specify rounding where needed
Immutable list result	`List.copyOf` on supported JDKs	Check supported Java version first

Handy Java skeleton:

```
static int solve(int[] input) {
    Objects.requireNonNull(input);
    // State or data structure.
    // Loop invariant.
    for (int value : input) {
        // Update state.
    }
    return 0; // Replace with the required result.
}
```

In a coding interview, imports can usually be stated or written once: `import java.util.*;`. Avoid hiding the entire solution in a stream if you cannot explain its behavior and complexity. Mention if the solution relies on Java 8, 17, or 21 features.

Java pitfalls that commonly break otherwise sound algorithms

- `==` on objects compares references; use `equals` for logical value equality.
- A custom map key needs compatible `equals` and `hashCode`, and its key fields should remain stable after insertion.
- `int` arithmetic can wrap. Use `long` when summing or subtracting potentially large integers.
- A comparator based on `a - b` can overflow. Use `Integer.compare(a,b)`.
- `ArrayList.remove(int)` removes by index; `remove(Object)` removes by value.
- `PriorityQueue` iteration is not sorted; repeated `poll()` gives priority order.
- `String.charAt` returns a UTF-16 code unit. Clarify whether the prompt needs Unicode code points or human-visible characters.
- `BigDecimal.equals` considers scale; `compareTo` compares numeric value. Keep currency in the grouping key.
- `Map.get(key) == null` cannot distinguish missing key from a stored null; use `containsKey` if null is allowed.
- A thread-safe collection does not make a sequence of business actions atomic.

8. Practice ladder and answer checkpoints

Solve from a blank editor. The existing Java workbook has runnable reference solutions for problems 1-10. Problems 11-14 have checkpoint hints below; try them before reading the hints.

#	Problem	Time	Checkpoint
1	Two Sum	20 min	Hash map; duplicate values use different indices
2	Group anagrams	25 min	Frequency or sorted key; define alphabet
3	Longest substring without repeats	30 min	Sliding window; left boundary never moves backward
4	Merge closed intervals	30 min	Sort; max end preserves nested interval
5	First occurrence in sorted array	20 min	Binary search continues left after finding target
6	Balanced brackets	20 min	Stack; reject early close and leftover open
7	Top K frequent categories	30 min	Count map + heap; define tie order
8	Binary tree level order	25 min	BFS queue; capture size per level
9	LRU cache	35 min	Map + linked list, or explain access-order LinkedHashMap
10	Successful totals by account and currency	25 min	BigDecimal; reject mixing currencies; clarify deduplication
11	Count subarrays whose sum equals K	35 min	Prefix sum counts; works with negative values
12	Validate a binary search tree	30 min	Propagate lower and upper bounds
13	First nonrepeating character	20 min	Count, then scan in original order
14	Shortest path in an unweighted graph	35 min	BFS; record visited when enqueueing

Hints and small proof checks for 11-14

11. Count subarrays with sum K: Let `prefix` be the sum before or at the current position. A prior prefix equal to `prefix - k` forms a subarray of sum k. Start the frequency map with `{0:1}` so subarrays starting at index 0 count. Increment the answer by the number of prior required prefixes, then record the current prefix. Expected O(n) time and O(n) space. Sliding window is unreliable when negative values are allowed.

12. Validate BST: Each node needs a range inherited from all ancestors, not just a comparison with its immediate parent. Recurse with exclusive lower and upper bounds; use `long` bounds for `int` node values. A left descendant greater than the root invalidates the tree even if it is less than its direct parent. O(n) time and O(h) stack space for height h.

13. First nonrepeating character: Count all character occurrences, then scan the original string and return the first whose count is one. Clarify what to return if none exists. O(n) time and O(alphabet) map space.

14. Shortest unweighted path: BFS visits nodes by increasing number of edges from the start. Mark a node visited when placing it in the queue, otherwise it may be enqueued many times. Store parent links if you need the path itself; store distances if you only need length. O(V + E) time and O(V) space. Directed versus undirected edges must be clear.

Payment-flavored coding prompt for a senior engineer

"Given payment events with tenant, account, currency, amount, status, transaction ID, and timestamp, return successful totals by tenant/account/currency. Duplicate transaction IDs must be counted once."

Clarify: Is transaction ID unique globally or only within a tenant? Can duplicate events have different amounts? Does a reversal have a separate ID or update the original? Is input ordered? What should happen with malformed records?

Good in-memory approach: Use a set keyed by tenant and transaction ID to accept only the first valid event, then a map keyed by tenant/account/currency with `BigDecimal` totals. This assumes the first event for a duplicate ID is authoritative. If conflicting duplicates are possible, reject or reconcile them under a stated policy. Expected O(n) map work and O(n) space. If this is a production service across instances, a durable unique constraint or provider-level idempotency mechanism is also needed.

Say: "I am keeping the tenant in both the deduplication key and the total key. I would not combine USD and EUR without a conversion rule. For an interview exercise, an in-memory set is enough; for a payment service, I would require durable coordination."

9. Two timed mock sessions

Mock A: algorithm, 45 minutes

Prompt: "Find the longest substring with at most K distinct characters." Use the worked example only **after** your first attempt.

Minute	Action
0-4	Clarify K, null, empty string, and character model
4-7	Trace `"eceba"`, K=2 and `"abc"`, K=0
7-11	State sliding-window invariant and complexity
11-30	Code without looking at a solution
30-38	Trace manually and test edge cases
38-45	Explain correctness, cost, and one limitation

Mock B: backend coding, 45 minutes

Prompt: "Aggregate successful payment records by account and currency, handling duplicate transaction IDs."

Minute	Action
0-5	Clarify tenant, duplicate conflict, amounts, status and return format
5-9	Work through six records including a duplicate, failure and two currencies
9-29	Write the solution with clear key types
29-37	Test duplicate conflict, empty input, decimal precision and currency split
37-45	Explain correctness, complexity and production consistency boundary

Use a real timer. Do not pause it to read the answer. After the mock, record what actually slowed you down: recognizing the pattern, Java syntax, edge cases, or explanation. Repair that specific weakness in study hour 19.

A five-point scorecard

Score 0-2 for each item: **contract clarity, suitable algorithm, correct implementation, meaningful tests, and clear explanation**. A score of 2 means you did it independently and accurately; 1 means you needed a prompt or missed a small case; 0 means it was absent or incorrect. A total of 8/10 with no zero is a useful practice target, not a hiring score.

10. Professional English while coding

Use the structure **decision -> reason -> check**. Speak in short sentences. A pause to think is normal.

Opening: "I'll clarify the input and output, trace one example, and then choose a data structure."

After choosing an approach: "A map lets me look up the value I need while scanning once. I am trading extra memory for lower expected time."

Before a complicated loop: "The invariant is that the current window has no more than K distinct characters. If it becomes invalid, I move the left boundary until it is valid again."

When you spot an error: "I see that this fails for a duplicate at the current index. I'll change the order so I check before inserting, then retest that case."

When uncertain: "I have not used that exact API recently. I know the algorithm and can write the simple loop, then verify the method name if needed."

Closing: "This is O(n) time because each element enters and leaves the window at most once. The frequency map uses space proportional to the number of distinct characters in the window."

Avoid memorizing these as fixed scripts. Practice saying the same idea in your own clear words. When you do not know an edge-case policy, ask or state an assumption. If you cannot finish, explain the partial algorithm, the missing step, and how you would test it.

Final 10-minute oral drill

1. Why does checking the map before insertion matter in Two Sum?
2. What is the invariant in a sliding-window solution?
3. Why is merge intervals O(n log n)?
4. When does binary search apply, and what is the first-occurrence change?
5. How do you test a heap comparator with ties?
6. Why can `int` arithmetic give the wrong answer?
7. What data structures give O(1) LRU operations?
8. Why must financial totals be grouped by currency?
9. What does an in-memory duplicate set fail to guarantee across services?
10. Which one problem will you repeat from a blank editor before the interview?

11. How to use the companion files

The runnable answer key is **InterviewPractice.java** in the same outputs folder. It contains all ten core implementations and 38 executable checks. Try each exercise first, then compare your approach, complexity, edge handling, and Java details. The main **interview-preparation.pdf** contains the 20-hour schedule and broader backend concepts. This companion occupies its coding blocks; it does not add eight hours to the schedule.

Use the language in this guide to explain your own reasoning. The target is a correct solution you can defend and adapt when the interviewer changes the prompt.